Acceptance Test Log

Test Environment

Tests are executed on the project's containerized stack: REST API, MySQL database, PostgreSQL database, MongoDB database and frontend, launched with Docker. Before acceptance testing, you must have:

- a MySQL test database, a PostgreSQL test database and a MongoDB test database containing a known dataset (for example, a table with a few rows);
- at least two user test accounts to verify data isolation;
- a recent browser with developer tools, and access to the application database for security checks.

1. Deployment and Containerization

Project objective: containerization (API, MySQL database, PostgreSQL database, MongoDB database, frontend)

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
DEP-01	**Starting the full stack**	Docker installed, repository cloned	1. Run "docker compose up -d" 2. List containers with "docker compose ps"	All 5 containers (API, MySQL, PostgreSQL, MongoDB) are started without errors or crash loops.		✓ OK ☐ KO
DEP-02	**Interface access**	Stack started	1. Open the frontend URL in the browser 2. Observe the Homepage	The Homepage displays without errors in the browser console.		✓ OK ☐ KO
DEP-03	**Frontend / API Communication**	Stack started	1. Open the Homepage 2. Observe network requests (developer tools)	Frontend requests to the API return a 2xx status code.		☐ OK ☐ KO

2. US00 - Authentication and account management

Description: create an account and log in securely so that only authorized administrators can access SAFEBASE

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US00-01	**Account creation**	Login page accessible, email not yet registered	1. Open the login page 2. Choose "Create an account" 3. Enter a valid email and password 4. Submit	The account is created and the user can access the application (redirected to the Homepage or the login page).		✓ OK ☐ KO
US00-02	**Email already registered**	An account exists for this email	1. Try to create an account with the same email	An error message is displayed, no duplicate account is created.		✓ OK ☐ KO
US00-03	**Invalid or missing fields**	Account creation form open	1. Leave the email or password empty, or enter a malformed email 2. Submit	A validation message is displayed, no account is created.		✓ OK ☐ KO
US00-04	**Login with valid credentials**	Registered account	1. Open the login page 2. Enter the correct email and password 3. Submit	The credentials are verified and the user is redirected to the Homepage.		✓ OK ☐ KO
US00-05	**Incorrect password**	Registered account	1. Enter the correct email and an incorrect password 2. Submit	An error message is displayed, the user stays on the login page, no session is created.		✓ OK ☐ KO
US00-06	**Unknown email**	No account for this email	1. Enter an unregistered email and any password 2. Submit	An error message is displayed. The message does not reveal whether the email exists.		✓ OK ☐ KO
US00-07	**Logout**	User logged in	1. Click "Log out" 2. Use the browser's back button	The session is closed, the user is redirected to the login page and cannot access protected pages.		✓ OK ☐ KO
US00-08	**Protected pages without session**	User not logged in	1. Open the Homepage URL directly in the browser	Access is denied and the user is redirected to the login page.		✓ OK ☐ KO
US00-09	**Invalid or expired session**	User logged in	1. Delete or alter the session token (developer tools) 2. Reload the Homepage or perform an action	Access is denied, the user is redirected to the login page.		☐ OK ☐ KO

Note: the bcrypt hashing (SEC-02) and the API access without a token (SEC-04) are already covered in section "Security and Confidentiality".

3. US01 - Adding a database

Description: add a connection to a database so that SAFEBASE can access and back it up

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US01-01	**Adding a valid MySQL database**	Accessible MySQL test database, credentials known	1. Click on "Add a database" 2. Select MySQL 3. Enter valid name, host, port, username, password 4. Save	Connection is successfully tested, the database is saved and appears in the list.		✓ OK ☐ KO
US01-02	**Adding a valid PostgreSQL database**	Accessible PostgreSQL test database	1. Click on "Add a database" 2. Select PostgreSQL 3. Enter valid information 4. Save	The database is saved and appears in the list.		✓ OK ☐ KO
US01-03	**Adding a valid MongoDB database**	Accessible MongoDB test database (if supported)	1. Click on "Add a database" 2. Select MongoDB 3. Enter valid information 4. Save	The database is saved and appears in the list.		☐ OK ☐ KO
US01-04	**Database type selection**	User on the Homepage	1. Click on "Add a database" 2. Open the type dropdown list	MySQL, PostgreSQL, and MongoDB types are available.		✓ OK ☐ KO
US01-05	**Missing required fields**	Add database form open	1. Leave one or more required fields empty (e.g. host, username) 2. Save	A message indicates required fields, no database is added.		✓ OK ☐ KO
US01-06	**Invalid credentials**	Test database accessible	1. Enter an incorrect password 2. Save	A connection error message appears, the database is not added to the list.		✓ OK ☐ KO
US01-07	**Unreachable server**	No server running on the specified host / port	1. Enter an incorrect host or port 2. Save	An error message is displayed within a reasonable timeframe, the database is not added.		✓ OK ☐ KO
US01-08	**Password never displayed**	At least one database added	1. View the database list and details 2. Inspect API responses (developer tools)	The database password appears neither in the interface nor in the API responses.		☐ OK ☐ KO

4. US02 - Schedule automatic backups

Description: schedule backups so that a database is backed up automatically without manual action

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US02-01	**Set a backup frequency**	One database added	1. Select the database 2. Enter time and date (or frequency) 3. Save	The schedule is saved, the corresponding cron expression is created, and the schedule is active.		✓ OK ☐ KO
US02-02	**Scheduling two databases**	Two databases added	1. Schedule the first database at frequency A 2. Schedule the second database at frequency B	Each database keeps its own frequency without interference.		✓ OK ☐ KO
US02-03	**Execution at scheduled time**	Database scheduled every minute (for testing)	1. Wait 1 to 2 minutes 2. Open the database backup list	A backup is launched automatically and a file is created at the specified location.		✓ OK ☐ KO
US02-04	**Backup log recording**	One automatic backup completed	1. Open the backup log entry	The log entry contains at least the date, database, file location, and size.		✓ OK ☐ KO
US02-05	**MySQL and PostgreSQL backup**	One database of each type scheduled	1. Wait for execution 2. Verify generated files	A usable backup file (mysqldump / pg_dump) exists for each database.		✓ OK ☐ KO
US02-06	**Disable a schedule**	Active schedule	1. Disable automatic backup 2. Wait for scheduled time	No backup is executed, and configured frequency is retained.		✓ OK ☐ KO
US02-07	**Invalid cron expression**	Scheduling form open	1. Enter an invalid frequency 2. Save	An error message appears, no schedule is created.		✓ OK ☐ KO
US02-08	**Backup failure**	Database scheduled	1. Stop the database server before the scheduled time 2. Wait for scheduled time	The backup is logged with a failed status, and the failure is reported to the user.		✓ OK ☐ KO

5. US03 - View and manage backup versions

Description: view backup versions for a database to select the right version to restore

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US03-01	**Display database versions**	Multiple existing backups for a database	1. Select the database	All backups for this database are listed with date, size, and status (depending on the database type).		☐ OK ☐ KO
US03-02	**Association to the correct database**	Backups exist on two databases	1. Select database A, then database B	Each database displays only its own backups.		☐ OK ☐ KO
US03-03	**Distinguishing versions**	Multiple backups at different dates	1. Observe the list	Versions are easily distinguishable (date, time, size) and each has a unique log record.		☐ OK ☐ KO
US03-04	**Identify a failed backup**	At least one failed backup	1. Observe the list	The failed backup is clearly identified (status visible).		☐ OK ☐ KO
US03-05	**Restore a version**	Database with a successful backup, then data added after the backup	1. Click on the version clock icon 2. Confirm restore 3. Check database data	The database returns to the selected version's state (data added after the backup is gone).		☐ OK ☐ KO
US03-06	**Select an older version**	At least two backups with different states	1. Restore the oldest version 2. Verify data	The content of the chosen version is restored, not the latest one.		☐ OK ☐ KO

6. US04 - Restore a database backup

Description: restore a specific backup version to recover data after a loss or error

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US04-01	**Select a specific version**	Database with at least two successful backups	1. Select the database 2. Choose one backup version 3. Click "Restore"	The restore dialog refers to the selected version (date, file).		☐ OK ☐ KO
US04-02	**Confirmation warning**	Backup version selected	1. Click "Restore" 2. Read the dialog	A confirmation dialog warns that current data will be replaced. Nothing is restored before the user confirms.		✓ OK ☐ KO
US04-03	**Cancel the restore**	Confirmation dialog displayed, data modified since the backup	1. Click "Cancel" 2. Check the database data	No restore is started and the current data is unchanged.		✓ OK ☐ KO
US04-04	**MySQL restore**	MySQL test database backed up, then data modified	1. Restore the backup and confirm 2. Check the data 3. Check the API logs	Data is back to the backup state, and the restore was run with the MySQL tool (mysql / mysqldump format).		✓ OK ☐ KO
US04-05	**PostgreSQL restore**	PostgreSQL test database backed up, then data modified	1. Restore the backup and confirm 2. Check the data 3. Check the API logs	Data is back to the backup state, and the restore was run with the PostgreSQL tool (psql / pg_restore).		✓ OK ☐ KO
US04-06	**MongoDB restore**	MongoDB test database backed up, then data modified	1. Restore the backup and confirm 2. Check the data 3. Check the API logs	Data is back to the backup state, and the restore was run with the MongoDB tool (mongorestore).		☐ OK ☐ KO
US04-07	**Success feedback**	Valid backup	1. Restore the backup and confirm 2. Observe the interface	A success message is displayed to the user once the restore is complete.		☐ OK ☐ KO
US04-08	**Failure feedback and alert**	Backup whose file was deleted (or target database stopped)	1. Restore this version and confirm 2. Observe the interface and the alerts	A failure message is displayed to the user and a restore failure alert is created.		✓ OK ☐ KO

Note: restoring an older version rather than the latest one is already covered by US03-06.

7. US05 - Remove a database connection

Description: remove a database connection to clean up databases that are no longer managed

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US05-01	**Confirmation before deletion**	At least one database in the list	1. Select a database 2. Click "Delete"	A confirmation is requested and the database is still in the list until the user confirms.		☐ OK ☐ KO
US05-02	**Delete a database**	Confirmation dialog displayed	1. Confirm the deletion	The database is removed from the list.		☐ OK ☐ KO
US05-03	**Cancel the deletion**	Confirmation dialog displayed	1. Click "Cancel"	The database remains in the list, with its configuration unchanged.		☐ OK ☐ KO
US05-04	**Deletion persisted**	Database deleted	1. Reload the page 2. Log out and log in again	The deleted database no longer appears in the list. .		☐ OK ☐ KO
US05-05	**Source database untouched**	Database added, containing a known dataset	1. Delete the connection in SAFEBASE 2. Connect directly to the database server	Only the SAFEBASE connection is removed, the database and its data remain intact on the server.		☐ OK ☐ KO
US05-06	**Other databases unaffected**	Two databases added, with backups and schedules	1. Delete database A 2. Observe database B	Database B remains in the list with its backups and its schedule.		☐ OK ☐ KO
US05-07	**Schedule of a deleted database**	Database with an active schedule (every minute for testing)	1. Delete the database 2. Wait 1 to 2 minutes	No more automatic backups are launched for this database (the cron job is removed).		✓ OK ☐ KO
US05-08	**Existing backups after deletion**	Database with several existing backups	1. Delete the database 2. Check the backup list and the backup files	The behaviour matches the decision taken (backups kept or deleted), consistently between the interface, the log records and the files. *(To be completed once the decision is made.)*		✓ OK ☐ KO

8. US06 - View and manage alerts

Description: be alerted when a backup or restore fails to react quickly to problems

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US06-01	**Alert on failed backup**	Database scheduled	1. Stop the database server before the scheduled time 2. Wait for the scheduled time 3. Open the alerts	The backup fails and an alert is automatically created for this backup.		✓ OK ☐ KO
US06-02	**No alert on success**	Database scheduled, server running	1. Wait for a successful backup 2. Perform a successful restore 3. Open the alerts	No alert is created for the successful operations.		✓ OK ☐ KO
US06-03	**Alert list display**	At least one alert created	1. Open the alerts section	Each alert is visible with the concerned database and the date.		☐ OK ☐ KO
US06-04	**Read / unread distinction**	One read alert and one unread alert	1. Observe the alert list	Read and unread alerts are clearly distinguishable. .		☐ OK ☐ KO
US06-05	**Alert details for troubleshooting**	Alert created	1. Open the alert	The details (operation type, database, date and time, cause of the error when known) are enough to start troubleshooting.		☐ OK ☐ KO
US06-06	**Alerts on several databases**	Two databases, a failure on each	1. Trigger a failure on database A and on database B 2. Open the alerts	Two distinct alerts are created, each one linked to the right database.		✓ OK ☐ KO

9. US07 - Alert on restore failure

Description: receive alerts when a restore fails to react quickly

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
US07-01	**Alert creation**	A backup whose file was deleted (or target database stopped)	1. Start restore for this version	Restore fails and a restore_failed alert is created, linked to the affected backup.		✓ OK ☐ KO
US07-02	**Alert content**	Alert created	1. Open the alert	The alert displays the database name, file, date and time of failure, and root cause if known.		☐ OK ☐ KO
US07-03	**"Unread" indicator**	Alert created	1. Perform an action in the application 2. Observe the indicator	The alert is visible with an "unread" indicator.		☐ OK ☐ KO
US07-04	**Mark as read**	Unread alert	1. Mark alert as read	The "unread" indicator disappears.		☐ OK ☐ KO
US07-05	**Alert history**	Alert marked as read	1. View alert history	The read alert remains viewable.		☐ OK ☐ KO
US07-06	**User isolation**	Two user accounts, one alert for account A	1. Log in with account B 2. View alerts	Account B sees no alerts linked to account A's databases.		☐ OK ☐ KO

10. Security and Confidentiality

Target skills: encrypted or hashed sensitive data, secure server components

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
SEC-01	**Encrypted database password**	One database added	1. Inspect corresponding row in the application database	The database password is not stored in plain text (encrypted value).		✓ OK ☐ KO
SEC-02	**Hashed user password**	One user account created	1. Inspect user row in the application database	The password is stored as a hash (bcrypt) and not in plain text.		☐ OK ☐ KO
SEC-03	**Accessing another user's databases**	Two accounts, one database for account A	1. With account B, call API endpoints for account A's database (list, backups, restore)	Access is denied (403 or 404), no data for account A is returned.		☐ OK ☐ KO
SEC-04	**Unauthenticated access**	User not logged in	1. Call a protected API route without a token	Access is denied (401).		☐ OK ☐ KO

11. User Interface

Target skills: interface conforms to mockups, responsive design

ID	Test case	Prerequisites	Steps	Expected result	Actual result	Status
UI-01	**Mockup compliance**	Approved mockup available	1. Compare each screen (Homepage, add, schedule, versions, alerts) with the mockup	Screens match the mockup (structure, labels, UI elements).		☐ OK ☐ KO
UI-02	**Responsive display**	Browser with responsive mode	1. View application on mobile, tablet, and desktop viewports	The interface remains readable and usable across all screen sizes.		☐ OK ☐ KO

Acceptance Summary

Section	Tests	OK	KO	Not executed
1. Deployment and containerization	3			
2. US00 - Authentication and account management	9			
3. US01 - Adding a database	8			
4. US02 - Schedule automatic backups	8			
5. US03 - View and manage backup versions	6			
6. US04 - Restore a database backup	8			
7. US05 - Remove a database connection	8			
5. US06 - View and manage alerts	6			
9. US07 - Alert on restore failure	6			
10. Security and confidentiality	4			
11. User interface	2			
Total	**37**			

Acceptance decision: ☐ Passed ☐ Passed with reservations ☐ Failed

Comments:

Date and tester signature: